Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Катышев М.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 17.10.26

Москва, 2026

Постановка задачи

Вариант 9.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы. В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить. Рассчитать детерминант матрицы (используя определение детерминанта)

Общий метод и алгоритм решения

Использованные системные вызовы:

- pthread_create(pthread_t *thread, const pthread_attr_t *attr, void (start_routine) (void), void arg) — создает новый поток внутри текущего процесса. Принимает указатель на переменную для сохранения идентификатора потока, атрибуты (обычно NULL), указатель на функцию, с которой начнется выполнение потока, и аргумент, передаваемый в эту функцию. Используется для запуска рабочих потоков, вычисляющих миноры матрицы.

- pthread_join(pthread_t thread, void **retval)** — блокирует выполнение главного потока до тех пор, пока указанный поток не завершит свою работу. Позволяет дождаться окончания вычислений и безопасно освободить ресурсы потока. Используется в конце программы для сбора результатов от всех запущенных потоков.

- pthread_mutex_lock(pthread_mutex_t mutex) и *pthread_mutex_unlock(pthread_mutex_t mutex) — функции блокировки и разблокировки мьютекса (взаимной блокировки). Мьютекс гарантирует, что только один поток в конкретный момент времени имеет доступ к общим данным. Используются для безопасного изменения счетчика активных потоков (current threads), предотвращая состояние гонки (race condition).

- pthread_cond_wait(pthread_cond_t cond, pthread_mutex_t mutex) — переводит поток в режим ожидания (усыпляет его) до тех пор, пока не будет получен сигнал по условной переменной. Автоматически освобождает мьютекс на время сна и захватывает его обратно при пробуждении. Используется, когда лимит одновременно работающих потоков исчерпан.

- pthread_cond_signal(pthread_cond_t cond) — посылает сигнал о пробуждении одному из потоков, находящемуся в ожидании на условной переменной. Используется в конце работы потока, чтобы разбудить следующий ожидающий поток и передать ему рабочее место.

- clock_gettime(clockid_t clk_id, struct timespec tp) — получает текущее время указанных часов. Используется с флагом CLOCK_MONOTONIC (монотонные часы, которые не зависят от изменения системного времени) для точного замера времени выполнения последовательного и параллельного алгоритмов.

Сначала была написана последовательная программа. В ней реализованы функции динамического выделения памяти под матрицу (double), её освобождения и рекурсивная функция вычисления детерминанта через разложение по первой строке. В main добавили чтение размера матрицы из аргументов командной строки, генерацию случайных чисел с фиксированным seed (для одинаковых данных в обоих тестах) и замер времени через clock_gettime. Затем на основе этого кода была создана параллельная программа. В неё добавили глобальные переменные для доступа из потоков, структуру для передачи данных в поток и функцию thread_func. В этой функции реализовали механизм пула: с помощью мьютекса и условной переменной ограничили количество одновременно работающих потоков (чтобы не создавать их больше, чем ядер). Каждый поток вычисляет свой минор и сохраняет результат. В main организовали цикл создания потоков (pthread_create) и цикл ожидания их завершения (pthread_join) с последующим суммированием результатов. После отладки обе программы были скомпилированы с флагами оптимизации и протестированы на матрице 11x11 с разным числом потоков. Результаты времени были занесены в таблицу, на основе которой рассчитаны ускорение и эффективность.**

Код программы

determinant_seq.cpp

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

double** create_matrix(int n){
   double** matrix = (double**)malloc(n * sizeof(double*));
   for (int i = 0; i < n; i++) {
      matrix[i] = (double*)malloc(n * sizeof(double));
   }
   return matrix;
}

void free_matrix(double** matrix, int n) {
   for (int i = 0; i < n; i++) {
   free(matrix[i]);
   }
   free(matrix);
}

double determinant(double** matrix, int n) {
   if (n == 1) return matrix[0][0];
   if (n == 2) return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];

   double det = 0;
   double** minor = create_matrix(n - 1);

   for (int col = 0; col < n; col++) {
      for (int i = 1; i < n; i++) {
         int minor_col = 0;
         for (int j = 0; j < n; j++) {
            if (j == col) continue;
minor[i - 1][minor_col++] = matrix[i][j];
         }
```

```
        }

double sign = (col % 2 == 0) ? 1.0 : -1.0;
det += sign * matrix[0][col] * determinant(minor, n - 1);
    }

    free_matrix(minor, n - 1);
    return det;
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        printf("Usage: %s <matrix_size>\n", argv[0]);
        return 1;
    }

    int n = atoi(argv[1]);

    double** matrix = create_matrix(n);
    srand(42);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            matrix[i][j] = (rand() % 10) + 1.0;

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    double det = determinant(matrix, n);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_nsec - start.tv_nsec) / 1000000.0;

    printf("Sequential Det: %f\n", det);
    printf("Time: %.3f ms\n", time_taken);

    free_matrix(matrix, n);

    return 0;
}
```

determinant_par.cpp

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <pthread.h>

double** global_matrix;
int global_n;
int max_threads;
int current_threads = 0;

pthread_mutex_t thread_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t thread_cond = PTHREAD_COND_INITIALIZER;

double** create_matrix(int n){
    double** matrix = (double**)malloc(n * sizeof(double*));
```

```
    for (int i = 0; i < n; i++) {
        matrix[i] = (double*)malloc(n * sizeof(double));
    }
    return matrix;
}

void free_matrix(double** matrix, int n) {
    for (int i = 0; i < n; i++) {
    free(matrix[i]);
    }
    free(matrix);
}

double determinant(double** matrix, int n) {
    if (n == 1) return matrix[0][0];
    if (n == 2) return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];

    double det = 0;
    double** minor = create_matrix(n - 1);

    for (int col = 0; col < n; col++) {
        for (int i = 1; i < n; i++) {
            int minor_col = 0;
            for (int j = 0; j < n; j++) {
                if (j == col) continue;
                minor[i - 1][minor_col++] = matrix[i][j];
            }
        }

        double sign = (col % 2 == 0) ? 1.0 : -1.0;
        det += sign * matrix[0][col] * determinant(minor, n - 1);
    }

    free_matrix(minor, n - 1);
    return det;
}

typedef struct {
    int col;
    double result;
} ThreadData;

void* thread_func(void* arg) {
    ThreadData* data = (ThreadData*)arg;

    pthread_mutex_lock(&thread_mutex);
    while (current_threads >= max_threads) {
        pthread_cond_wait(&thread_cond, &thread_mutex);
    }

    current_threads++;
    pthread_mutex_unlock(&thread_mutex);

    double** minor = create_matrix(global_n - 1);
    int col = data->col;
    for (int i = 1; i < global_n; i++) {
        int minor_col = 0;
        for (int j = 0; j < global_n; j++) {
```

```
        if (j == col) continue;
        minor[i - 1][minor_col++] = global_matrix[i][j];
        }
    }

    double sign = (col % 2 == 0) ? 1.0 : -1.0;

    data->result = sign * global_matrix[0][col] * determinant(minor, global_n - 1);

    free_matrix(minor, global_n - 1);
    pthread_mutex_lock(&thread_mutex);
    current_threads--;
    pthread_cond_signal(&thread_cond);
    pthread_mutex_unlock(&thread_mutex);

    return NULL;
}

int main(int argc, char* argv[]) {

    if (argc != 3) {
        printf("Usage: %s <matrix_size> <max_threads>", argv[0]);
        return 1;
    }

    global_n = atoi(argv[1]);
    max_threads = atoi(argv[2]);

    global_matrix = create_matrix(global_n);
    srand(42);
    for (int i = 0; i < global_n; i++)
        for (int j = 0; j < global_n; j++)
            global_matrix[i][j] = (rand() % 10) + 1.0;

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_t* threads = (pthread_t*)malloc(global_n * sizeof(pthread_t));
    ThreadData* tdata = (ThreadData*)malloc(global_n * sizeof(ThreadData));

    for (int col = 0; col < global_n; col++) {
        tdata[col].col = col;
        tdata[col].result = 0;
        pthread_create(&threads[col], NULL, thread_func, &tdata[col]);
    }

    double det = 0;
    for (int col = 0; col < global_n; col++) {
        pthread_join(threads[col], NULL);
        det += tdata[col].result;
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_nsec - start.tv_nsec) / 1000000.0;

    printf("Parallel Det: %f\n", det);
    printf("Time: %.3f ms\n", time_taken);
```

```
    free(threads);
    free(tdata);
    free_matrix(global_matrix, global_n);

    return 0;
}
```

Протокол работы программы

Тестирование:

Для получения среднего времени выполнения программы для каждого случая выполняется 10 запусков. После откидываются результаты выполнения программы с самым быстрым и медленным временем выполнения и по оставшимся данным вычисляется среднее время выполнения. Все программы работают я квадратными матрицами размерности 11.

```
./seq 11
Sequential Det: 5121452632.000000
Time: 717.197 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 830.234 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 733.522 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 854.888 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 740.080 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 721.216 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 838.940 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 901.260 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 839.782 ms
./seq 11
Sequential Det: 5121452632.000000
Time: 736.723 ms

./par 11 1
Parallel Det: 5121452632.000000
Time: 924.259 ms
./par 11 1
Parallel Det: 5121452632.000000
Time: 835.068 ms
./par 11 1
Parallel Det: 5121452632.000000
Time: 821.383 ms
./par 11 1
Parallel Det: 5121452632.000000
Time: 815.163 ms
./par 11 1
Parallel Det: 5121452632.000000
Time: 1202.453 ms
```

./par 11 1
Parallel Det: 5121452632.000000
Time: 864.063 ms
par 11 1
Parallel Det: 5121452632.000000
Time: 969.167 ms
./par 11 1
Parallel Det: 5121452632.000000
Time: 924.492 ms
./par 11 1
Parallel Det: 5121452632.000000
Time: 867.489 ms
./par 11 1
Parallel Det: 5121452632.000000
Time: 1262.825 ms

./par 11 2
Parallel Det: 5121452632.000000
Time: 508.362 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 627.827 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 717.332 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 468.097 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 429.026 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 473.743 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 627.540 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 637.322 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 428.671 ms
./par 11 2
Parallel Det: 5121452632.000000
Time: 524.521 ms

./par 11 3
Parallel Det: 5121452632.000000
Time: 417.382 ms
./par 11 3
Parallel Det: 5121452632.000000
Time: 381.185 ms
./par 11 3
Parallel Det: 5121452632.000000
Time: 461.199 ms
./par 11 3
Parallel Det: 5121452632.000000
Time: 562.774 ms
./par 11 3
Parallel Det: 5121452632.000000
Time: 488.693 ms
./par 11 3
Parallel Det: 5121452632.000000
Time: 317.082 ms
./par 11 3
Parallel Det: 5121452632.000000

Time: 316.015 ms
./par 11 3
Parallel Det: 5121452632.000000
Time: 499.894 ms
./par 11 3
Parallel Det: 5121452632.000000
Time: 281.622 ms
par 11 3
Parallel Det: 5121452632.000000
Time: 334.388 ms

./par 11 10
Parallel Det: 5121452632.000000
Time: 463.916 ms
./par 11 10
Parallel Det: 5121452632.000000
Time: 313.427 ms
./par 11 10
Parallel Det: 5121452632.000000
Time: 395.614 ms
./par 11 10
Parallel Det: 5121452632.000000
Time: 306.751 ms
./par 11 10
Parallel Det: 5121452632.000000
Time: 299.124 ms
./par 11 10
Parallel Det: 5121452632.000000
Time: 298.292 ms
./par 11 10
Parallel Det: 5121452632.000000
Time: 341.510 ms
./par 11 10
Parallel Det: 5121452632.000000
Time: 388.761 ms
./par 11 10
Parallel Det: 5121452632.000000
Time: 390.856 ms
par 11 10
Parallel Det: 5121452632.000000
Time: 481.662 ms

./par 11 128
Parallel Det: 5121452632.000000
Time: 362.146 ms
./par 11 128
Parallel Det: 5121452632.000000
Time: 270.439 ms
./par 11 128
Parallel Det: 5121452632.000000
Time: 302.223 ms
./par 11 128
Parallel Det: 5121452632.000000
Time: 368.688 ms
./par 11 128
Parallel Det: 5121452632.000000
Time: 302.916 ms
./par 11 128
Parallel Det: 5121452632.000000
Time: 413.317 ms
./par 11 128
Parallel Det: 5121452632.000000
Time: 375.732 ms
./par 11 128
Parallel Det: 5121452632.000000
Time: 287.930 ms
./par 11 128

Parallel Det: 5121452632.000000
Time: 386.181 ms
./par 11 128
Parallel Det: 5121452632.000000
Time: 391.040 ms

par 11 1024
Parallel Det: 5121452632.000000
Time: 309.674 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 298.948 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 342.170 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 288.187 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 323.369 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 279.357 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 396.893 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 394.626 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 539.163 ms
./par 11 1024
Parallel Det: 5121452632.000000
Time: 279.063 ms

Таблица

Число потоков	Время исполнения (мс)	Ускорение	Эффективность (%)
Последовательно	**786.375**	**1**	**100**
1	**888.25**	**0.8853**	**88.53**
2	**546.625**	**1.4386**	**71.93**
3	**397.25**	**1.9795**	**65.98**
10	**361.875**	**2.173**	**21.73**
128	**349.375**	**2.2508**	**1.75**
1024	**328.625**	**2.3929**	**0.23**

Вывод

В ходе выполнения работы были проанализированы результаты замеров, представленные в таблице. Можно выделить несколько ключевых закономерностей:

1. **Накладные расходы (1 поток). Параллельная версия с 1 потоком работает медленнее чисто последовательной (888 мс против 786 мс, ускорение 0.88). Это объясняется накладными расходами на создание потока (pthread_create), его завершение (pthread_join), а также затратами на захват и освобождение мьютекса, которые отсутствуют в последовательной программе.**

2. **Зона эффективного распараллеливания (2-3 потока). При увеличении числа потоков до 3 (что совпадает с количеством выделенных логических ядер среды выполнения) наблюдается существенное ускорение (время падает до 397 мс, ускорение почти в 2 раза). Каждый поток получает собственное физическое ядро, и потоки работают действительно параллельно.**
3. **Деградация после превышения количества ядер (10-1024 потока). Дальнейшее увеличение числа потоков не дает кратного ускорения. Время выполнения падает незначительно (с 361 мс до 328 мс), а ускорение упирается в предел (около 2.39)**
4. **Падение эффективности. Эффективность при 1024 потоках падает до 0.23%. Физически одновременно могут работать только 3 ядра. Остальные 1021 поток ожидают своей очереди на условной переменной. Операционная система вынуждена постоянно переключать контекст (context switch) между тысячами потоков, тратя на это ресурсы процессора. Время, затрачиваемое на синхронизацию, начинает превышать время полезной работы.**

Таким образом, для данной вычислительной системы оптимальным является использование 3 потоков (по числу ядер). Дальнейшее увеличение потоков нецелесообразно, так как приводит к огромным накладным расходам и падению эффективности.